Project Report:
Shared Compiler (DSL)

Team members:
Eldho : 24BAI0268
Hari : 24BAI0262
Soujanya : 24BCI0135
Pragadeeswar : 24BCE0618

1. Project Objective

The primary goal of this project is to develop a custom Domain-Specific Language (DSL) that dramatically simplifies graphics programming. By abstracting away the complexities of C/C++ memory management, pointers, and manual library initialization, this system allows users to render complex scenes using simple, English-like syntax. To achieve this, we built a custom source-to-source compiler (transpiler) that translates our high-level `.my` scripts into standard, optimized C code, which is then natively compiled using a custom SDL2 backend graphics engine.

2. Completed Milestones (Current Progress)

We have successfully developed a fully functional, end-to-end compiler pipeline that reads custom syntax, generates C code, and automatically compiles native executable graphics applications.

A. Language Design & Syntax Definition We designed a lightweight, beginner-friendly DSL capable of issuing procedural drawing commands. The current lexer and syntax parser successfully recognize and process the following language features:

- Initialization: `window [width] [height]` to define the rendering environment.
- State Management: `color [r] [g] [b]` to dynamically change the active rendering color.
- Geometric Primitives:
    - `circle [x] [y] [radius]`
    - `rectangle [x] [y] [width] [height]`
    - `line [x1] [y1] [x2] [y2]`

    - `triangle [x1] [y1] [x2] [y2] [x3] [y3]`
- Automated Mathematics: `center_rect [width] [height]` which offloads coordinate centering math to the backend engine.

B. The C++ Compiler Architecture We built the core translation engine in C++. It operates through three distinct phases:

- Lexical Analyzer (Lexer): Scans the raw text of the `.my` source file, strips whitespace, and tokenizes the characters into recognized grammatical components (e.g., `TOKEN_WINDOW`, `TOKEN_RECTANGLE`, `TOKEN_NUMBER`).
- Parser & Abstract Syntax Mapping: Validates the sequential logic of the tokens, ensuring that each command is paired with the correct number of integer parameters.
- Code Generator: Traverses the parsed data and outputs a completely new `generated_game.c` file, accurately mapping the custom syntax to pure C function calls.

C. The SDL2 Graphics Runtime Engine Instead of forcing the compiler to write complex hardware-level instructions, we developed a modular backend engine (`graphics_runtime.h`).

- This C header acts as the API between the generated code and the operating system.
- It securely handles the initialization of the SDL2 hardware renderer, manages the window lifecycle, and executes complex pixel-rendering algorithms (such as the Midpoint Circle Algorithm).

D. Automated Build Toolchain To create a seamless developer experience, we automated the final compilation stage. Once the C++ compiler finishes generating the `.c` file, it issues a system-level command to GCC. It automatically links the SDL2 libraries (`-lSDL2`) and builds the final executable application, allowing the user to go from a text script to a running graphics window in a single command.

4. Future Implementations & Project Roadmap

The current prototype successfully handles static 2D scene rendering. The future trajectory of this project aims to transition the system from a static drawing tool into a fully interactive, three-dimensional game development framework.

Phase 3: Interactivity and State Management (2D Movement)

- Variable Support: Upgrading the lexer and parser to recognize variable declarations (e.g., `player_x = 100`) rather than relying exclusively on hardcoded integers.
- Game Loop Generation: Modifying the code generator to support a `update` block, allowing the generated C code to redraw the screen 60 frames per second rather than rendering a single static frame.
- Event Handling: Introducing syntax to capture keyboard and mouse input (e.g., `if key_w pressed`), translating these into SDL_PollEvent logic in the C backend.

Phase 4: Advanced Game Logic

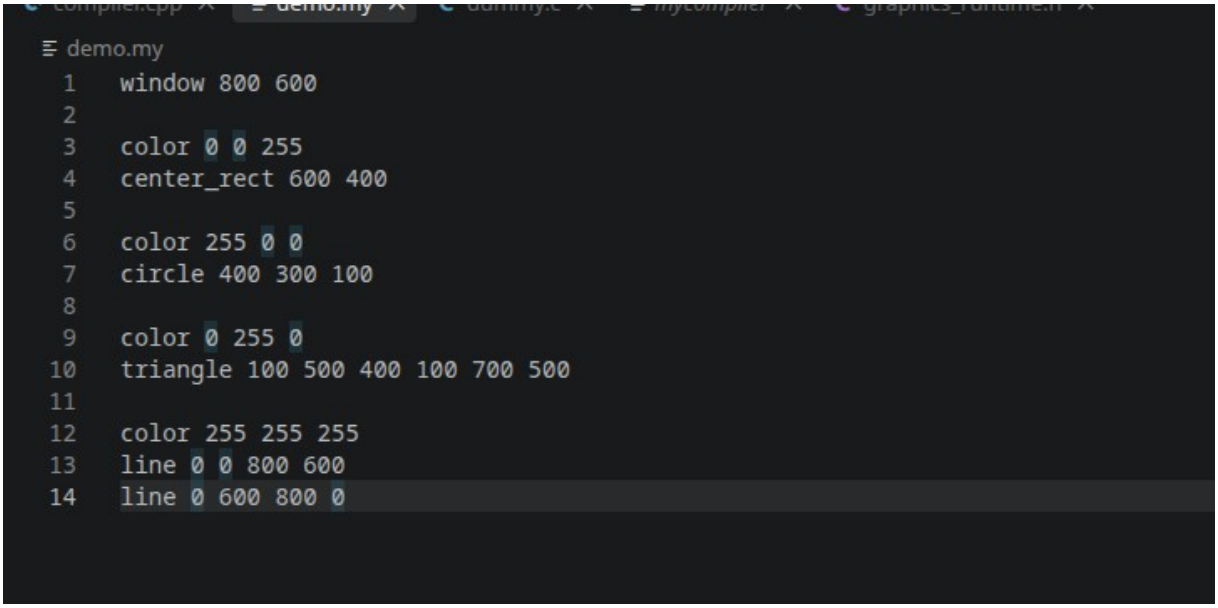
- Physics Abstractions: Adding high-level commands for basic physics, allowing users to apply gravity or velocity to objects without writing complex math.
- Collision Detection: Implementing backend C functions that calculate boundary intersections between generated shapes.

Phase 5: 3D Graphics Integration

- OpenGL Backend Swap: Replacing the 2D SDL2 rendering functions in `graphics_runtime.h` with an OpenGL (or similar 3D hardware API) rendering pipeline.
- 3D Primitives: Expanding the DSL syntax to include Z-axis coordinates and 3D objects (e.g., `cube`, `sphere`).

- Camera & Lighting Matrices: Abstracting the complex linear algebra required for 3D viewports into simple DSL commands (e.g., `camera [x] [y] [z]`).

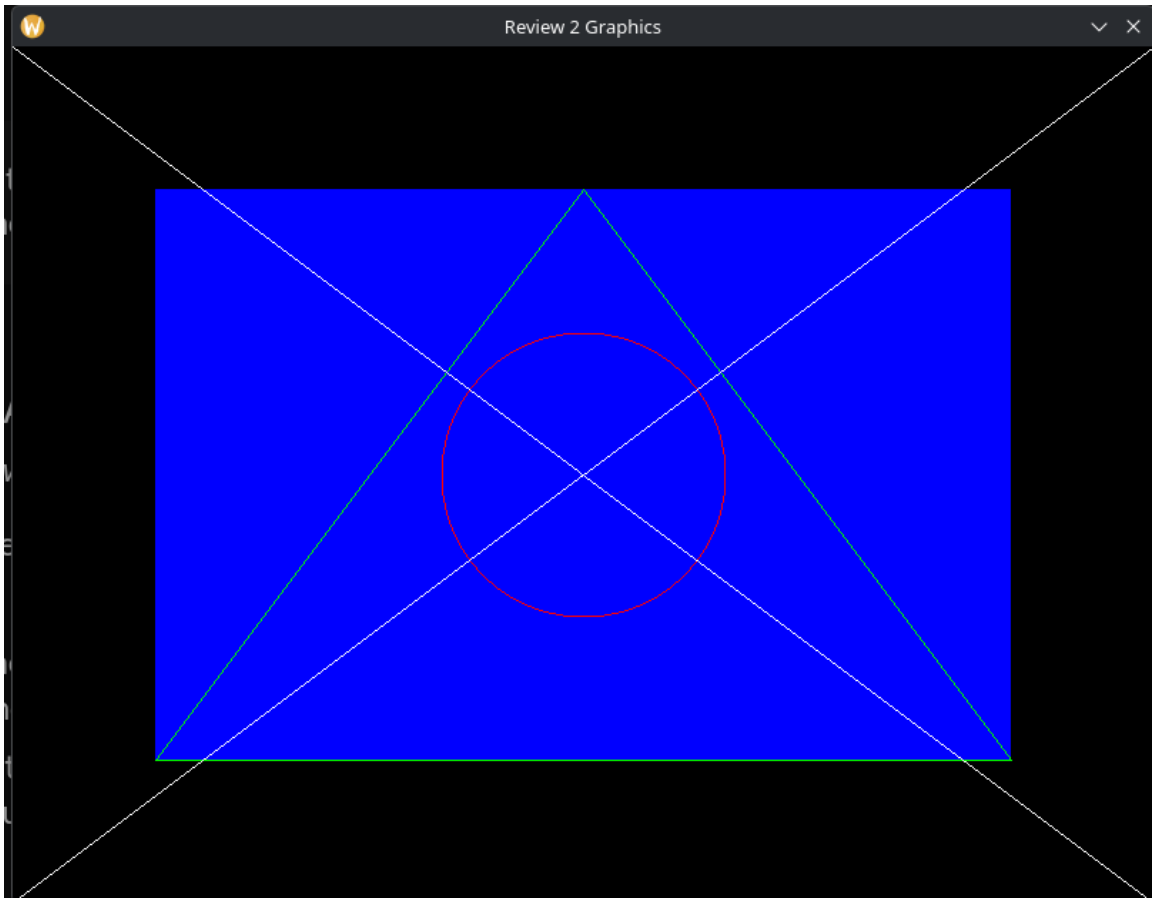
Implementations
1. The Custom Language Script



The frontend user script written in our custom Domain-Specific Language (DSL). What we built here:

- Abstracted Complexity: We designed a syntax that entirely removes the need for memory management, pointers, or library initialization.
- Procedural Logic: We structured the language to execute top-down, allowing users to define window constraints, set color states, and draw geometric primitives using simple sequential arguments.
- Proof of Concept: This file proves the compiler can handle multiple distinct commands (shapes, colors, centering) and varying argument lengths in a single execution pass.

2.The Output



What it is: The compiled native binary executable running on the operating system.
What we achieved here:

- Hardware Execution: This proves the final binary successfully initializes the system's graphics hardware via our SDL2 wrapper without crashing or leaking memory.
- State Machine Validation: The fact that the shapes render in different colors (e.g., a red circle on top of a blue rectangle) proves our generated C code correctly maintains and updates the rendering state sequentially.

- Algorithmic Verification: The perfectly centered graphics prove that the background algebraic calculations performed by our C engine successfully interpreted the global window dimensions at runtime.